



LAU

Lebanese American University
Chartered in the State of New York
www.lau.edu.lb

SOE

School of Engineering
*Department of Electrical &
Computer Engineering*
www.lau.edu.lb/ece



BYBLOS CAMPUS

P.O. Box: 36
Byblos, Lebanon
Tel: +961 9 547 262
Fax: +961 9 546 262

BEIRUT CAMPUS

P.O. Box: 13-5053 Chouran
Beirut 1102 2801, Lebanon
Tel: + 961 1 786 456
Fax: +961 1 867 098

NEW YORK HEADQUARTERS

211 E. 46th Street
New York, NY 10017-2935, USA
Tel: +1 212 203 4333
Fax: +1 212 784 6597

Intelligent Data Processing and Applications

COE 543/743

Chapter 7 - Structure and Content Similarity

Joe M. TEKLI, PhD

Computer Software Engineering

Ch. 7. Structure and Content Similarity

Chapter Contents

1. Introduction
2. Traditional Information Retrieval (IR)
 - Feature Representation
 - IR Data Model
 - Term Indexing
 - Relevance Ranking
 - Data Selection Operators
3. Extending IR Techniques to Semi-structured Data
 - Node Indexing
 - Base Vector Indexing
 - Augmented Vector Indexing
 - Single & Multiple Category Retrieval
 - Term-in-Context Indexing
 - Matrix Model
4. Underlying Indexing Techniques

Ch. 7. Structure and Content Similarity

1. Introduction

- Semi-structured data similarity is one of the central topics in the *Information Retrieval (IR)* field
- While **TED** (Tree Edit Distance)-based approaches for comparing semi-structured data:
 - Are designed to handle **rigorously structured (data-centric) data**
 - Are **fine-grained**, and thus require sophisticated processing and time to execute
 - Main target: data **structure**
 - Applications: **Data warehousing, data integration**, and structural **clustering/classification**
- **IR**-based methods for comparing semi-structured data:
 - Are designed to handle **loosely structured** (document-centric) data
 - Are **coarse-grained**, and thus require less processing time to execute
 - Mainly target: data **structure and content**
 - Application: **ranked querying**

Ch. 7. Structure and Content Similarity

1. Introduction

- Documents with **traditional IR** consist of **free (flat) text**
 - ⇒ Traditional IR methods are not adequate for handling semi-structured data
- Several attempts have been made to extend IR methods
 - Toward **semi-structured data** processing
 - Research area titled: **Semi-structured IR**
- Objectives of this chapter:
 - Describe basic concepts of **traditional IR**
 - Introduce main **semi-structured IR techniques**

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

- IR is a branch of informatics that deals with the **acquisition, organization, storage, search** and **retrieval** of **loosely structured/unstructured** information
- In essence, the goal of *IR* is to efficiently **identify/retrieve**, in a data collection, **information that is relevant** w.r.t. the user's needs
- The notion of **relevance** in *IR* is **broad** and **imprecise**
 - ⇒ Generally concretized by the notion of **similarity**
 - Comparing a user query with a set of documents
 - Comparing documents together

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

- Comparing IR and DB (Database) technologies

IR		DB
Loosely structured/unstructured	Data model	Rigorously structured
Natural language (e.g., keywords)	Query model	Structured (e.g., SQL)
Probabilistic	Search model	Deterministic
Approximate (Partial)	Matching Process	Exact (total)
Ranked based on their relevance	Query results	Unordered (arbitrary)
Non-expert	User expertise	Proficiency in querying formalism
Not needed	User knowledge	Need for DB schema

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

- **Free-form text**

- No predefined structure
- Very difficult to search and manage

- **Semi-structured data**

- Free-form text that is marked up with elements
- Relatively easier to search and manage

- **Structured data**

- The data is strictly tagged
- Easiest to search and manage

Formats

txt, doc, docx, pdf

XML, JSON,
and **RDF**-based
e.g., SBML, ProML,
CDA, LexRDF, Bio2RDF,

Usually adopted on the Web

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

- **Free-form text**
 - *n-gram* based features: character-based, term-based
 - *Word*-based features: syntactic, POS, semantic
- **Semi-structured** (and structured) text
 - *Structure-only*: tag-based, edge-based, path-based, tree-based
 - *Structure-and-content*: augmented vector, context vector
 - *Content-only*: same as free-form text features

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.1. N-Gram

- ***n*-gram**: a contiguous sequence of n items from a given text
 - 1-gram is a *unigram*, 2-gram is a *bigram*, 3-gram is a *trigram*, etc.

Examples	Unit	Sample sequence	1-gram sequence	2-gram sequence	3-gram sequence
Natural language	Term-based	my name is John	<my, name, is, john>	<my name, name is, is John>	<my name is, name is John>
Natural language	Character-based	my_name_is_John	<m, y, _, n, a, m, e, _, i, s, _, J, o, h, n>	<my, y_, _n, na, am, me, e_, _i, is, s_, _J, Jo, oh, hn>	my_, y_n, _na, nam, ame, me_, e_i, _is, is_, s_J, _Jo, Joh, ohn
Protein sequence	Amino acid (term-based)	Cys-Gly-Leu-Ser-Trp	<Cys, Gly, Leu, Ser, Trp>	<Cys-Gly, Gly-Leu, Leu-Ser, Ser-Trp>	<Cys-Gly-Leu, Gly-Leu-Ser, Leu-Ser-Trp>
DNA sequence	Base pair (character-based)	AGCTTCG	<A, G, C, T, T, C, G>	<AG, GC, CT, TT, TC, CG>	<AGC, GCT, CTT, TTC, TCG>

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.1. N-Gram

- ***n*-gram**: a contiguous sequence of n items from a given text
 - 1-gram is a *unigram*, 2-gram is a *bigram*, 3-gram is a *trigram*, etc.
- *n*-grams are processed in their original **sequence** form if **order** is relevant
 - Or can be represented as **un-ordered vectors**
 - For instance:

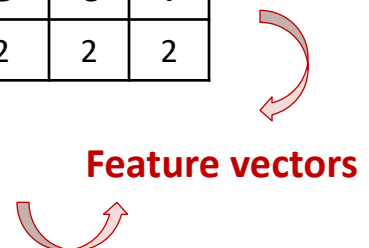
- 1-gram <A, G, C, T, T, C, G> represented as a vector:

A	G	C	T
1	2	2	2

- 3-gram <AGC, GCT, CTT, TTC, TCG>

represented as a vector:

AGC	GCT	CTT	TTC	TCG
1	1	1	1	1



Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2. *Word-based - Syntactic*

- **Word-based:** syntactic
 - Processing words occurrences in a sentence
 - Process: Tokenization, stop-word removal, stemming
- Example: let's say we want to encode two sentences
 - S1= A Byblos driver is frequently aggressive
 - S2 = Byblos players frequently shoot from three-points

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2. *Word-based - Syntactic*

Example:

1. **Tokenization:** identifying the separate tokens in an input sentence
 - S1= <A, Byblos, driver, is, frequently, a, little, aggressive>
 - S2 = <Byblos, players, frequently, shoot, from, three, points>
2. **Stop-word removal:** removing prepositions and meaningless words, e.g., *the, a, of, to*,
 - S1= <A, Byblos, driver, is, frequently, a, little, aggressive>
 - S2 = <Byblos, players, frequently, shoot, from, three, points>
3. **Stemming:** reducing inflected or derived words to their original form (stem)
 - S1= <Byblos, driver, **be, frequent**, little, aggressive>
 - S2 = <Byblos, **player, frequent**, shoot, three, **point**>

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2 Word-based - Syntactic

– **Word-based:** syntactic

- Sentences are processed in their original **sequence** form if **order is relevant**
 - Or can be represented as **un-ordered vectors**

- S1= <Byblos, driver, be, frequent, aggressive>

Byblos	driver	be	frequent	little	aggressive
1	1	1	1	1	1

- S2 = <Byblos, player, frequent, shoot, three, point>

Byblos	player	frequent	shoot	three	point
1	1	1	1	1	1

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2 Word-based - POS

- **Word-based:** POS (part of speech)
 - It allows to distinguish words based on their grammar groups
 - Such as: nouns (NN), proper nouns (NNP), verbs (VB), adjectives (JJ) , adverbs (RB), and determinants (DT)
- For instance, considering S1 = A Byblos driver is frequently aggressive
 - Its POS tagging becomes: A Byblos driver is frequently aggressive
 - ↑ ↑ ↑ ↑ ↑ ↑
 - DT NNP NN VBZ RB JJ**
- Usually represented as: A[DT] Byblos[NNP] driver [NN] is [VBZ] frequently[RB] aggressive[JJ]

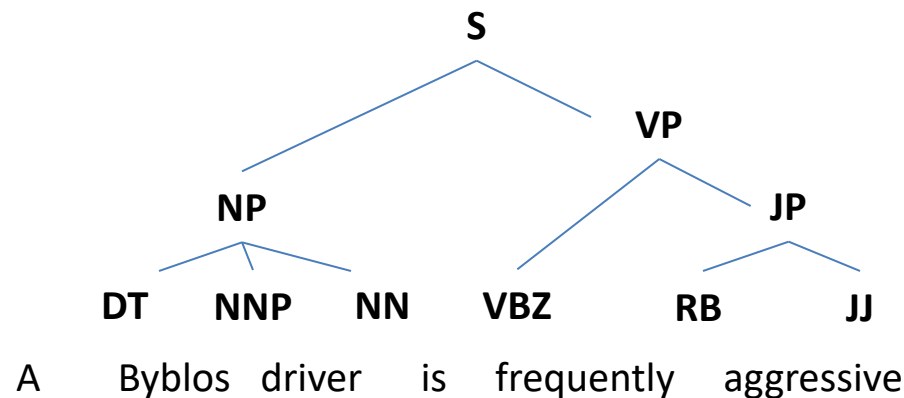
Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2 Word-based - POS

- **Word-based:** POS (part of speech)
 - A richer representation of a POS tagged sentence is the **parse tree**
 - Highlighting the grammar structure of the sentence



NP: Noun phrase, VP: verb phrase, JP: Gerund phrase

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2 Word-based - Semantic

- **Word-based:** Semantic

- Allows describing the meanings of words given their semantic definitions and relationships in a **reference lexical knowledge base** (dictionary, cf. Chapter 8) like **WordNet** (for general texts) or **UMLS** (for medical texts)

- For instance:

- S1= A **Byblos** driver is frequently aggressive

→ City of Byblos

→ Byblos basket ball team

- S2 = **Byblos** players frequently shoot from three-points

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.1. Feature Representation

2.1.2 Word-based - Semantic

- Every word can have multiple possible meanings
 - Word-based semantic representation consists of associating every word with its proper meaning in its textual context
 - Through a process dubbed: **word sense disambiguation** (cf. Chapter 9)

- For instance: A Byblos driver is frequently aggressive

1. **City of Byblos** ✓

2. Byblos basket ball team

1. **Operator of a motor vehicle** ✓

2. Someone who drives animals

3. Golfer

4. Computer program

1. Showing determination

2. Fast-growing

3. **Belligerent, eager to fight** ✓

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.2. IR Data Model

- Documents and user queries in IR usually consist of **sets of keywords**
- Identifying documents that are **relevant (similar)** to a query comes down to:
 - **Comparing** the **keywords** of each document, in the document collection, to those of the query
 - **Ranking** the documents, w.r.t. their keyword similarities, with the query
 - Document selection is undertaken using dedicated selection strategies
 - Defining a certain **threshold**

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.2. IR Data Model

- Various *IR* models have been proposed in the literature:
 - **Set-theoretic** models:
 - Boolean model [1]
 - Fuzzy Retrieval model [2]
 - **Algebraic** models:
 - **Vector Space Model (VSM)** [3]
 - Generalized (topic-based) VSM (GVSM) [4]
 - Latent Semantic Indexing (LSI) model [5]
 - **Probabilistic** models:
 - Probabilistic relevance model [6]
 - Divergence From Randomness (DFR) model [7]



We restrict ourselves to **VSM**

- Most commonly used
- High performance

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.2. IR Data Model

- Following the **Vector Space Model** (VSM):
 - A document **D** and user queries **Q** are both represented as **vectors** $\vec{V}_D$ and $\vec{V}_Q$
 - **Vector dimensions** represent each an indexing unit
 - An **indexing unit** stands for a single term
 - A **term** is a lexical word/expression
 - The coordinate of a given *document D* in vector $\vec{V}_D$ on dimension t_i , is noted $w_D(t_i)$
 - It stands for the weight of t_i in document **D** within a document collection
 - The relevance (similarity) of a doc **D** to a query **Q** is evaluated as $Sim(\vec{V}_D, \vec{V}_Q)$
 - Using a measure of similarity between vectors, e.g., cosine, Pearson correlation, etc.,

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.3. Term Indexing

- Weight $w_D(t_i)$ is usually computed using a **TF-IDF** score [8]
 - Which considers both document and collection statistics
 - With different mathematical variations
- The TF-IDF score comprises of two factors:
 - **TF**: Term Frequency
 - **IDF**: Inverse Document Frequency

$$w_D(t_i) = \text{TF}(t_i, D) \times \text{IDF}(t_i, C)$$

- Where: t_i is a term, D is a document, and C is the document collection

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.3. Term Indexing

- The **TF (Term Frequency)** factor designates the number of times a term t_i occurs in document D (document statistics)
 - The underlying idea is that the importance of a given term t_i in describing a document D increases with the frequent use of t_i in D
- The **IDF (Inverse Document Frequency)** factor indicates the fraction of documents that contain term t_i (collection statistics)
 - The underlying idea is that the importance of a given term t_i in describing a document D decreases with the frequent use of t_i in the document collection C
 - A term appearing in more documents becomes less distinctive/descriptive of a specific document
 - ⇒ IDF estimates the *rarity* of t_i in the document collection C

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.3. Term Indexing

- The **TF (Term Frequency)** factor designates the number of times a term t_i occurs in document D (document statistics)
 - Formally: $TF(t_i, D) = Freq_D(t_i) \in [0, |D|]$
- The **IDF (Inverse Document Frequency)** factor indicates the fraction of documents that contain term t_i (collection statistics)
 - Formally: $IDF(t_i, C) = \log \frac{N}{DF(t_i, C)} \in [0, \infty[$
 - Where: N is the number of documents in C , i.e., $N = |C|$
 $DF(t_i, C)$ is the number of docs in C containing at least one occurrence of term t_i

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.3. Term Indexing

- Example: Consider documents D_1 , D_2 , and D_3 . Compute TF-IDF vector weights considering the underlined terms as vector dimensions.

 D_1

LAU is an American university located in Lebanon. It is one of the leading American universities in Lebanon.

 D_2

USJ is a French university founded in Lebanon. It is a leading university in Lebanon.

 D_3

LMA: the Lebanese military academy is the only military university in Lebanon.

 $\vec{V}_{D1}$

<u>LAU</u>	<u>American</u>	<u>University</u>	<u>Leading</u>	<u>Lebanon</u>
$1 \times \log(3/1)$	$2 \times \log(3/1)$	$2 \times \log(3/3)$	$1 \times \log(3/2)$	$2 \times \log(3/3)$

 $\vec{V}_{D2}$

<u>USJ</u>	<u>French</u>	<u>University</u>	<u>Leading</u>	<u>Lebanon</u>
$1 \times \log(3/1)$	$1 \times \log(3/1)$	$2 \times \log(3/3)$	$1 \times \log(3/2)$	$2 \times \log(3/3)$

 $\vec{V}_{D3}$

<u>LAM</u>	<u>Lebanon</u>	<u>Military</u>	<u>Academy</u>	<u>University</u>
$1 \times \log(3/1)$	$2 \times \log(3/3)$	$2 \times \log(3/1)$	$1 \times \log(3/1)$	$1 \times \log(3/3)$

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.3. Term Indexing

- Various mathematical formulations exist for TF-IDF computations:

$$w_D(t_i) = \text{TF}(t_i, D) \times \text{IDF}(t_i, C)$$

TF(t_i, D)	IDF(t_i, C)	Explanation
$\text{Freq}_D(t_i) \in [0, D]$	$\log \frac{N}{\text{DF}(t_i, C)} \in [0, \infty[$	$w_D(t_i)$ is of null value when: TF=0 or IDF=0 and increases accordingly
$\frac{\text{TF}(t_i, D)}{\max(\text{TF}(D))} \in [0, 1]$	$\log \left(\frac{N}{\text{DF}(t_i, C)} \right) + 1 > 0$	$w_D(t_i)$ is of null value only when TF=0 (IDF is always >0). Also, $\text{TF} \in [0, 1]$
$\log (\text{TF}(t_i, D) + 1) \geq 0$	$ \log(\text{DF}(t_i, C)) \in [0, \infty[$	$w_D(t_i)$ is of null value when: TF=0 and IDF=0 , and increases accordingly. Yet, TF increases logarithmically

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.4. Relevance Ranking

- The relevance of a doc **D** to a query **Q** is evaluated by computing the similarity between corresponding document vectors
 - Formally: $Sim(\vec{V}_D, \vec{V}_Q)$
 - Using a measure of similarity between vectors
 - **Cosine**
 - **Pearson correlation coefficient (PCC)**
 - **Dice**
 - etc.



Refer to Chapter 6

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.5. Data Selection Operators

- Term **weights reflect** the **impact of keywords** in documents/queries
 - Document/query weight **vectors are** then **compared** to evaluate **relevance**
 - ⇒ **Results** (documents) are **ranked** following their relevance w.r.t. the query
- Two main strategies exist for selecting ranked results
 - **Range query operator**
 - **KNN** (K- Nearest Neighbor) **operator**

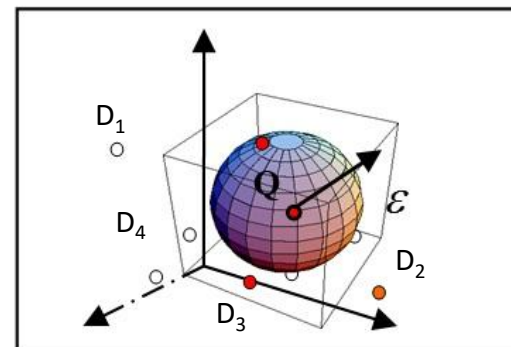
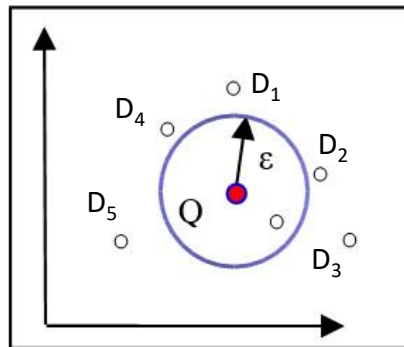
Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.5. Data Selection Operators

- **Range Query operator:**

- A range query operator θ_R returns the set of documents $\{D_i\}$ located within a certain range ϵ from query Q using a distance function d (inverse of similarity)
 - Any user specified document D_j can be used as the range center instead of Q
- Formally: $\sigma(Q, \theta_R, \epsilon) = \{\forall D_i / d(D_i, Q) \leq \epsilon / \epsilon \in \text{dom}(d)\}$



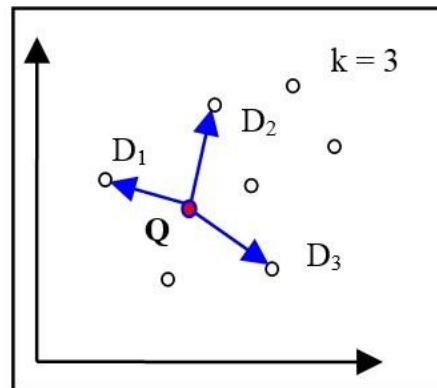
Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.5. Data Selection Operators

- **KNN operator:**

- A K-Nearest Neighborhood (KNN) operator θ_{KNN} returns the set of k neighboring documents located nearest to a query Q using a distance function d
- Formally: $\sigma(Q, \theta_{\text{KNN}}, k) = \{ \forall D_{i=1 \dots k} / d(D_i, Q) \leq d(D_i, D_m) \}$

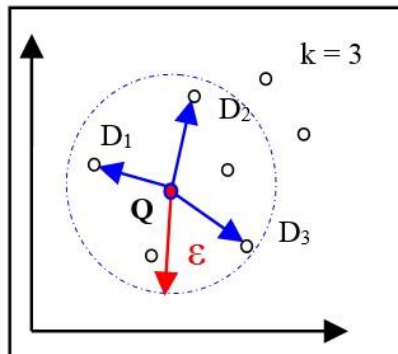


Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.5. Data Selection Operators

- **Combined range and KNN operator:**
 - A combined range and KNN operator θ_{R-KNN} returns the k nearest neighbors located within a certain range ϵ from a query Q using a distance function d
 - Formally: $\sigma(Q, \theta_{R-KNN}, \epsilon, k) = \{\forall V_{i=1\dots k} / d(D_i, Q) \leq d(D_i, V_m) \wedge d(D_i, Q) \leq \epsilon / \epsilon \in dom(d)\}$



Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.6. Evaluation

- Every **DB-style query** exactly defines a set of rows to be retrieved
 - Every row is an **exact/complete match** of the query: and thus a **correct answer**
 - ⇒ There is **never a bad hit** for a DB-style query
- An **IR-style query** **can return bad hits**
 - Documents in the hit list only **approximately/partially match** the query
 - The query does not provide a complete specification of the **user's intention**
 - Every query has an implied intention that **cannot be described fully** in the query and hence is not available to the retrieval engine



In **IR**, there is **no** general **way to** write a better query that **returns only relevant documents**

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.6. Evaluation

- Need for IR evaluation measures
 - Based on the idea of **benefit/cost** of a retrieved document to the user
 - A **good (relevant) hit** has a certain benefit to the user
 - A **bad (non-relevant) hit** has a certain cost to the user: time spent reading it, cost of getting the document translated, edited, and so on
- Main measures used:
 - **Precision:** percentage of **relevant documents** correctly **retrieved** by the system with respect to **all documents retrieved** by the system
 - **Recall:** percentage of **relevant documents** correctly **retrieved** by the system with respect to **all documents relevant** for the user

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.6. Evaluation

- More formally:

	Retrieved	Not Retrieved
Relevant	TP (True Positives)	FN (False Negatives)
Non-relevant	FP (False Positives)	TN (True Negatives)

Query result visualization

- Precision: $PR = \frac{TP}{TP + FP} \in [0,1]$ High *precision* denotes that the search engine achieved high accuracy, selecting documents that are relevant to the query
- Recall: $R = \frac{TP}{TP + FN} \in [0,1]$ High *recall* means that very few relevant documents were not retrieved by the search engine

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.6. Evaluation

- For instance, consider query q producing where the **expected result** is $\{a, b, c, d\}$

- If the search engine returns **actual results**: $\{a, x, b\}$

$$\text{Precision: } PR = \frac{2}{3}$$

$$\text{Recall: } R = \frac{2}{4}$$

	Retrieved	Not Retrieved
Relevant	2	2
Non-relevant	1	Don't care...

- If the search engine returns **actual results**: $\{a, x, y, z, b, c, d\}$

$$\text{Precision: } PR = \frac{4}{7}$$

$$\text{Recall: } R = \frac{4}{4} = 1$$

	Retrieved	Not Retrieved
Relevant	4	0
Non-relevant	3	Don't care...

- If the search engine returns **actual results**: $\{a\}$

$$\text{Precision: } PR = \frac{1}{1} = 1$$

$$\text{Recall: } R = \frac{1}{4}$$

	Retrieved	Not Retrieved
Relevant	1	3
Non-relevant	0	Don't care...

Ch. 7. Structure and Content Similarity

2. Traditional IR Techniques

2.6. Evaluation

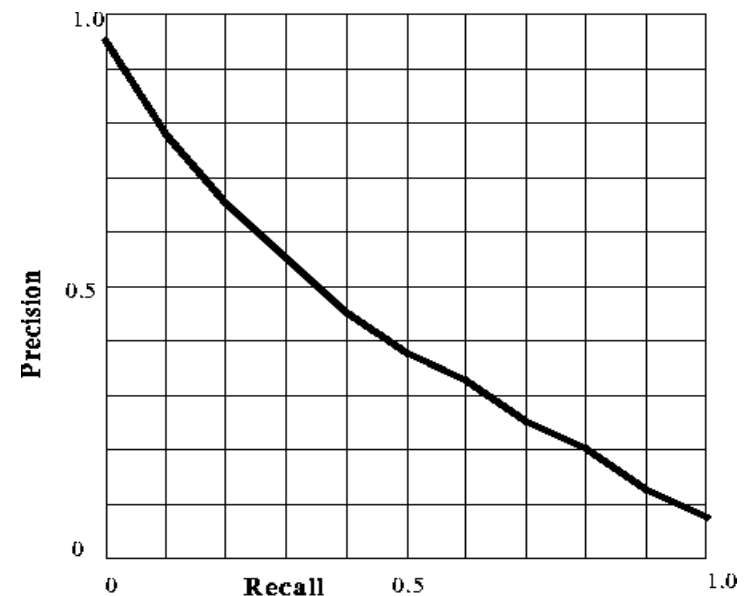
- Trying to **increase recall** typically introduces more bad hits into the hit list
 - Thereby **reducing precision** (i.e., moving to the right along the PR-R curve)
- Trying to **increase precision** removes some good hits from the hit list
 - Thereby **reducing recall** (i.e., moving left along the PR-R curve)
- Goal of a retrieval engine: to **increase both precision and recall**
- In addition to comparing precision and recall, it is a common practice to consider the **F-value** measure, as the harmonic mean of *precision* and *recall*:

$$\mathbf{F\text{-}value} = \frac{2 \times \mathbf{PR} \times \mathbf{R}}{\mathbf{PR} + \mathbf{R}} \in [0,1] \quad \text{High/low F-value means high/low retrieval quality}$$

2. Traditional IR Techniques

2.6. Evaluation

- **Precision-Recall graph:**
 - Evaluating the **ranking** of relevant results w.r.t. non-relevant ones in the query result list
 - Depicting the **degradation** of **precision at n** as one traverses the hit list by plotting interpolated precision numbers against **percentage recall**
 - A percentage recall of say 50% is the position in the hit list at which 50% of the relevant documents in the collection (i.e., $0.5 * R$) have been retrieved



It is a measure of the number of hits you have to read before you see a certain percentage of relevant documents

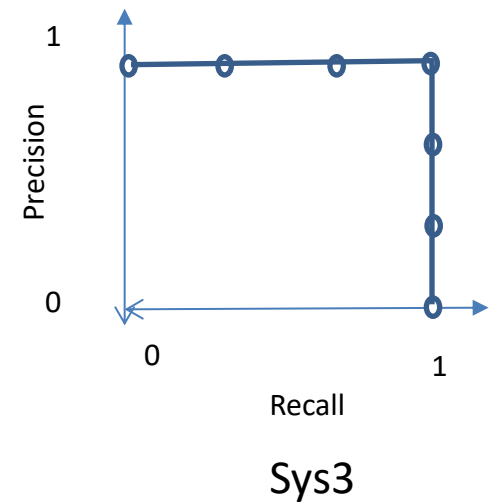
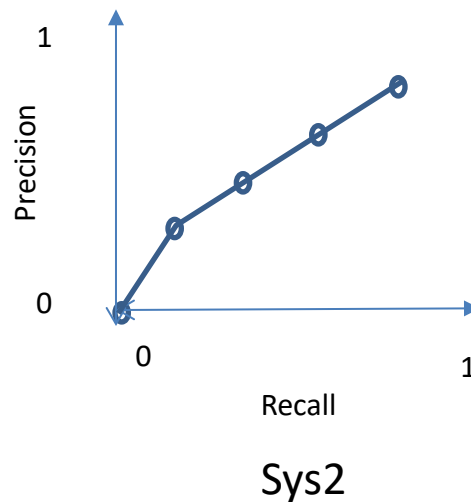
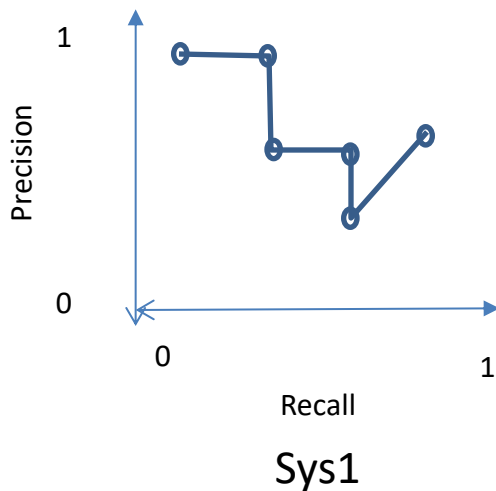
2. Traditional IR Techniques

2.6. Evaluation

- Example:** Consider a certain query Q applied on a given dataset $C = \{a, b, c, d, x, y\}$, where the query's expected results = $\{a, b, c, d\}$.

Consider three different search engines producing the below search results:

- Generated Results_{Sys1} = $\langle a, b, x, c, y, d \rangle$ PR = $4/6$ R = $4/4 = 1$
- Generated Results_{Sys2} = $\langle x, y, a, b, c, d \rangle$ PR = $4/6$ R = $4/4 = 1$
- Generated Results_{Sys3} = $\langle a, b, c, d, x, y \rangle$ PR = $4/6$ R = $4/4 = 1$



2. Traditional IR Techniques

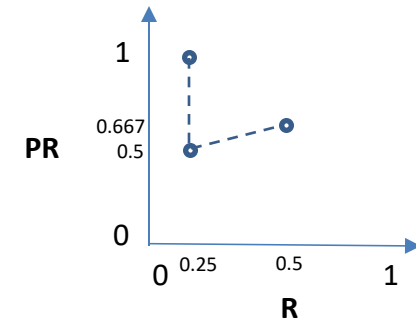
2.6. Evaluation

- For instance, consider query q producing where the **expected result** is $\langle a, b, c, d \rangle$

- Here, we take into account the order of the elements in the hit list

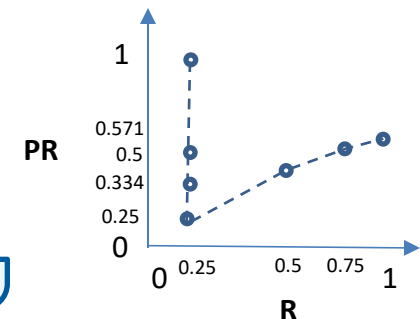
- If the search engine returns **actual results**: $\langle a, x, b \rangle$

At hit #1: PR = 1	Recall: R = 0.25
At hit #2: PR = 0.5	Recall: R = 0.25
At hit #3: PR = 0.667	Recall: R = 0.5



- If the search engine returns **actual results**: $\{a, x, y, z, b, c, d\}$

At hit #1: PR = 1	Recall: R = 0.25
At hit #2: PR = 0.5	Recall: R = 0.25
...	
At hit #7: PR = 0.571	Recall: R = 1



The **best** precision-recall **graph** curve moves up and out to the right so that both recall and precision are higher at every point along the curve

2. Traditional IR Techniques

2.6. Evaluation

- **Mean Average Precision (MAP):** a single measure to evaluate the ranking of relevant results w.r.t. non-relevant ones in the query result list

- Instead of analyzing PR/R graphs

$$\text{MAP} = \frac{\sum_{j=1..n} (\text{PR}[j] \times \text{rel}[j])}{N} \in [0, 1]$$

- n be the number of query hits, and $\text{PR}[j]$ is *precision* at hit j
 - $\text{rel}[j]$ is equal to 1 if the j th data point in the result list is relevant and 0 otherwise
 - $N = TP + FN$ is the total number of relevant data points in the data collection
- **MAP = 1** when the system retrieves all relevant data points (**recall=1**) and ranks them perfectly:
 - All of them appearing before non-relevant data points in the query list (**PR at Nth hit =1**)
 - It decreases as more non-relevant data points are introduced before relevant ones in the result list

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

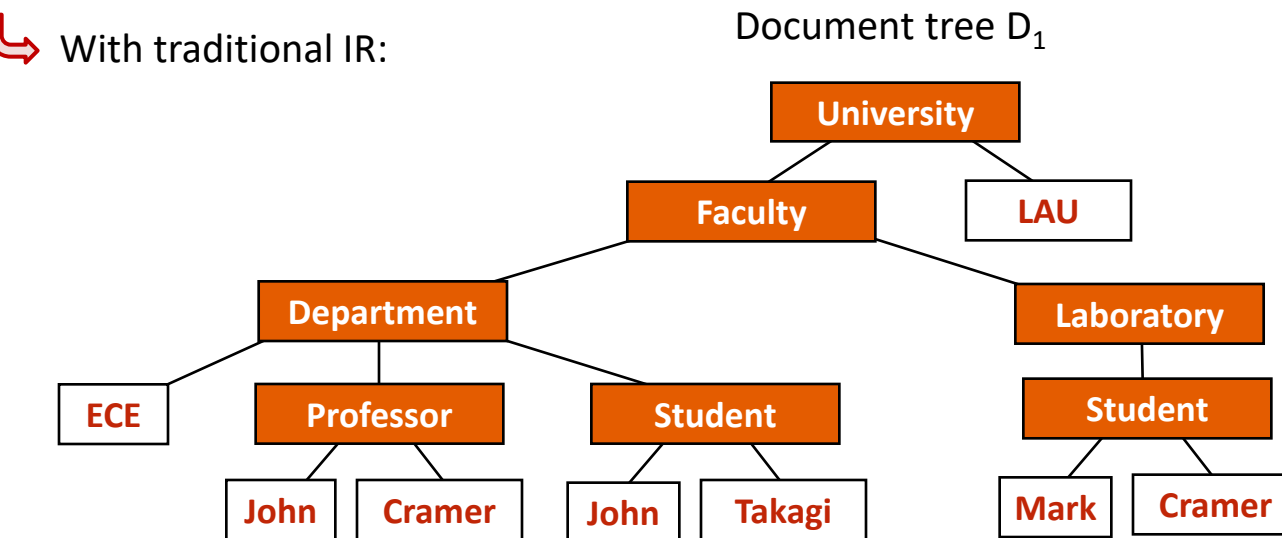
- Documents are flat (unstructured) with conventional information retrieval
 - ⇒ Traditional *IR* methods are not adequate with semi-structured data
 - Motivation 1:
 - Semi-structured content is distributed at **different structural levels** of the document tree
 - **Information** placed **near the root** node of an XML document **tends to be more important** than information further down in the hierarchy
 - Intuitively, as one descends in the document tree hierarchy
 - Information becomes increasingly specific, consisting of finer and finer details
 - Its effect on the meaning of the whole document tree decreases accordingly
- ↳ This is orthogonal to traditional flat documents where info is placed at **one single level** and is of the **same relevance**

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example:

↳ With traditional IR:



$\vec{V}_{D1}$

LAU	ECE	John	Cramer	Takagi	Mark
1	1	2	2	1	1



TF weights

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

- Documents are flat (unstructured) with conventional information retrieval
 - ⇒ Traditional *IR* methods are not adequate with semi-structured data
- Motivation 2:
 - **Users** in turn would like to **refer to document structure** when searching for relevant information in semi-structured documents
 - They pose so called *content-and-structure* queries, in comparison with the *content-only* queries in conventional *IR*
 - By restricting the context of interest to some elements in the documents being searched



Extending *IR* methods to account for the structure of Semi-structured documents in the similarity/comparison process

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.1. Indexing Nodes

- A semi-structured document is generally processed as a number of *indexing nodes* [9]
 - Atomic units in the document, i.e., elements (attributes), which encompass disjoint sub-trees
 - Each indexing node underlines a **distinct indexing context**
 - Such that TF-IDF **weights** can be **computed locally**, instead of being evaluated w.r.t. to the whole document
 - The weights would then be **augmented**, down-weighting the *TF-IDF* scores when the terms are **propagated upwards** in the document tree
 - ⇒ Thus taking into account the hierarchical aspect of semi-structured data

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.1. Indexing Nodes

- How to identify *indexing nodes*?
 1. Automatically by the system, following some criteria
 - For instance, all leaf nodes (or all element nodes containing data values) could be considered indexing nodes
 2. By the user (administrator), following a dedicated grammar definition (in which indexing nodes are designated) corresponding to the semi-structured document
 - E.g., hypothetical DTD element declaration **<!ELEMENT Professor (#INDEX)>** could designate that *Professor* elements in the document are indexing nodes

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.1. Indexing Nodes

- **Base vectors:** vectors associated to each of the indexing nodes
 - **Vector dimensions:** represent the indexing units of the vector
 - **Indexing units:** are the distinct terms in the node's textual value
 - **Weights:** represent the **TF-IEF** scores of terms within the node's textual value

Formally: $w_E(t_i) = TF(t_i, E) \times IEF(t_i, D)$

- Where: E is an element corresponding to an index node in document tree D

$TF(t_i, E) = Freq_E(t_i) \in [0, |E|]$ // $|E|$ is the number of terms in E 's textual content

$IEF(t_i, D) = \log \frac{N}{EF(t_i, D)} \in [0, \infty[$ // Inverse **Element Frequency**

- Where: N is the number of indexing nodes in D , i.e., $N = |D|$

$EF(t_i, D)$ is the number of indexing node elements in D containing t_i



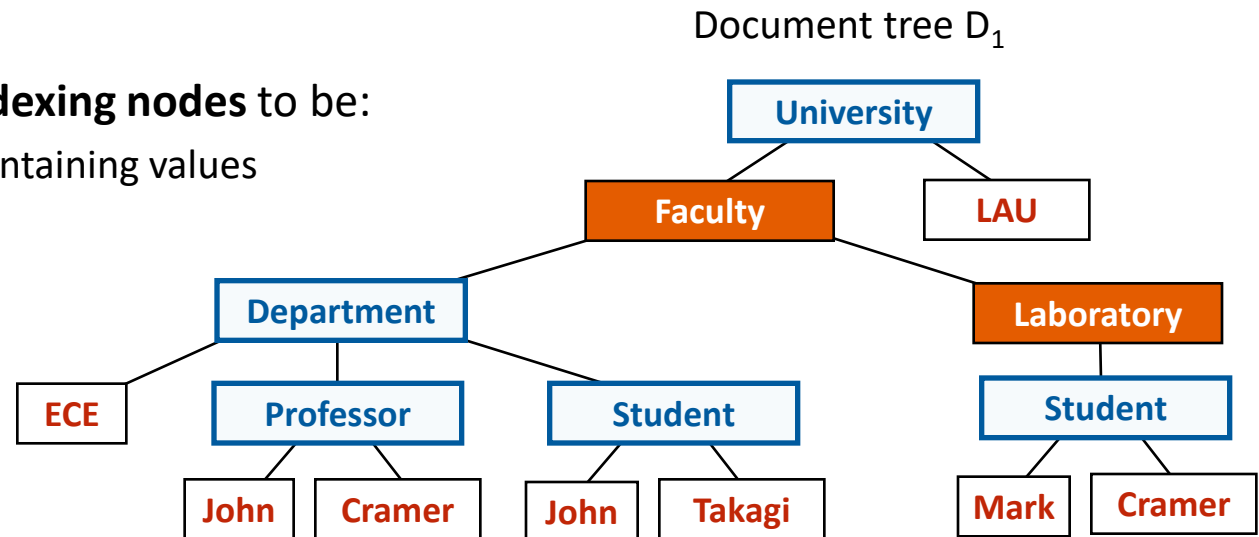
Other adapted **TF-IEF** formulas can be used (cf. Section 2.2)

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example: Consider **indexing nodes** to be:

- All element nodes containing values



All base vector dimensions:

	LAU	ECE	John	Cramer	Takagi	Mark
$\vec{V}_{Uni}$	1	0	0	0	0	0
$\vec{V}_{Dept}$	0	1	0	0	0	0
$\vec{V}_{Prof}$	0	0	1	1	0	0
$\vec{V}_{Stud1}$	0	0	1	0	1	0
$\vec{V}_{Stud2}$	0	0	0	1	0	1

We use TF-only weights
in the following
examples for ease
of presentation

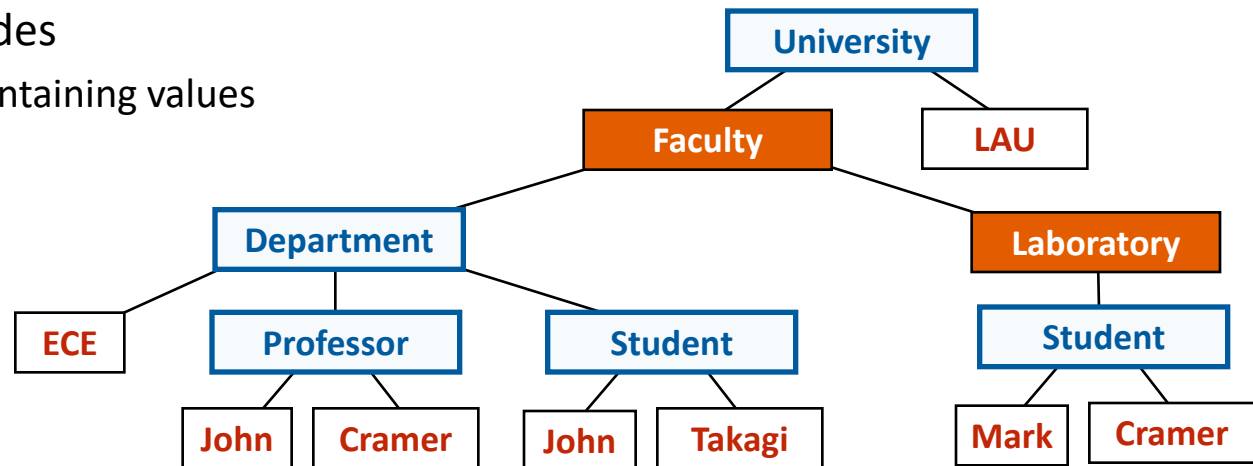
Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example: Indexing nodes

- All element nodes containing values

Document tree D1



All base vector dimensions:

Combining TF and IEF weights

	LAU	ECE	John	Cramer	Takagi	Mark
$\vec{V}_{Uni}$	$1 \times \log(5/1)$	0	0	0	0	0
$\vec{V}_{Dept}$	0	$1 \times \log(5/1)$	0	0	0	0
$\vec{V}_{Prof}$	0	0	$1 \times \log(5/2)$	$1 \times \log(5/2)$	0	0
$\vec{V}_{Stud1}$	0	0	$1 \times \log(5/2)$	0	$1 \times \log(5/1)$	0
$\vec{V}_{Stud2}$	0	0	0	$1 \times \log(5/2)$	0	$1 \times \log(5/1)$

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.2. Augmented Vectors

- Augmented vectors are computed for the ancestors of indexing nodes
 - By **down-weighting** the *TF-IDF* scores when the terms are **propagated upwards** in the document tree
 - ⇒ Thus taking into account the hierarchical aspect of semi-structured data
- Main premise: The **larger the distance** between a node and its ancestor, the **lesser the** node contributes to the **relevance** of its ancestor's content
 - Practically, we associate **structural weights** to node edges: α_j^i
 - α_j^i is the structural weight of the edge linking nodes E_i and E_j
 - Can be defined randomly/automatically by the system, following some criteria (e.g., following the *semantic similarity* of node labels)
 - Or can be chosen by domain experts



α_j^i is usually defined as the inverse of a distance

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.2. Augmented Vectors

- Computing augmented vector weights:
 - Given $\alpha_j^i \in [0, 1]$ representing a similarity (inverse of distance) function
 - We can define the weight of an element E in the augmented vector:

$$\mathbf{w}_{E_i}(\mathbf{t}_r) = \sum_{\mathbf{t}_r} \pi(\mathbf{w}_j^i) \times \mathbf{w}_{E_j}(\mathbf{t}_r)$$

$\text{Path}(E_i, E_j)$

- where $\pi(\mathbf{w}_j^i)$ underlines the product of structural weights of all edges $\in \text{Path}(E_i, E_j)$

E_j is an indexing node, and E_i one of its ancestor element nodes

Ch. 7. Structure and Content Similarity

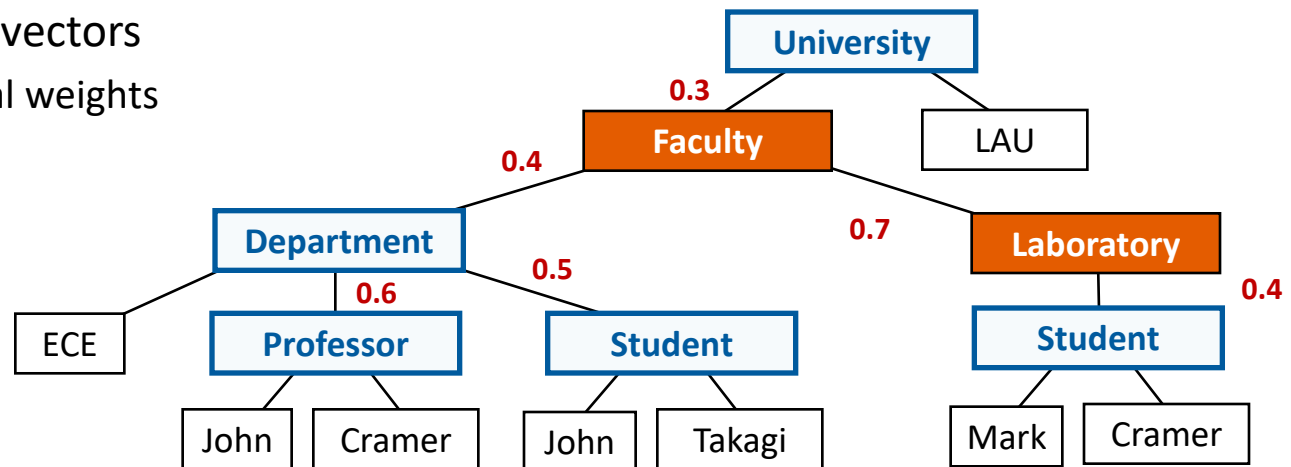
3. Extending IR Techniques to Semi-structured Data

Document tree D1

Example: Augmented vectors

- Considering structural weights

Considering the
base vectors
computed in the
previous slides,
with base TF-IDF
Weights: w_E



	LAU	ECE	John	Cramer	Takagi	Mark
$\vec{V}_{Lab}$	0	0	0	$w_{Cramer}^2 \times 0.4$	0	$w_{Mark} \times 0.4$
$\vec{V}_{Dept}$	0	w_{ECE}	$w_{John1} \times 0.6 + w_{John2} \times 0.5$	$w_{Cramer1} \times 0.6$	$w_{Takagi} \times 0.5$	0
$\vec{V}_{Fac}$	0	$w_{ECE} \times 0.4$	$w_{John1} \times 0.6 \times 0.4 + w_{John2} \times 0.5 \times 0.4$	$w_{Cramer1} \times 0.6 \times 0.4 + w_{Cramer2} \times 0.4 \times 0.7$	$w_{Takagi} \times 0.5 \times 0.4$	$w_{Mark} \times 0.4 \times 0.7$
$\vec{V}_{Uni}$	w_{LAU}	$w_{ECE} \times 0.4 \times 0.3$	$w_{John1} \times 0.6 \times 0.4 \times 0.3 + w_{John2} \times 0.5 \times 0.4 \times 0.3$	$w_{Cramer1} \times 0.6 \times 0.4 \times 0.3 + w_{Cramer2} \times 0.4 \times 0.7 \times 0.3$	$w_{Takagi} \times 0.5 \times 0.4 \times 0.3$	$w_{Mark} \times 0.4 \times 0.7 \times 0.3$

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.3. Single/Multi-category Weighting

- Users may want to refer to a given part (category) of the semi-structured document
 - Requesting only content that is relevant to that category
 - ⇒ **Single category retrieval**
 - The category is a sub-tree rooted at a certain document tree node
- Example: The user is interested in **students** who only **work in laboratories**
 - Thus, only the contents of nodes **Student** corresponding to category **Laboratory** should be considered when computing term statistics
 - The score for term “Cramer” – corresponding to student *Mark Cramer* – should not be affected by occurrences of “Cramer” in elements outside the category **Laboratory**

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.3. Single/Multi-category Weighting

- Users may request information from several categories
 - Or they do not care to which category the requested content belongs
 - ⇒ **Multi-category retrieval**
- Example: The user is interested in all **students**
 - Terms statistics should be computed taking into account all categories

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.3. Single/Multi-category Weighting

- Solution: Pre-compute weights for *indexing nodes*
 - Then derive augmented weights on-the-fly, i.e. at query runtime [10]
 - Taking into account user preferences/query scope



Flexible retrieval

- Augmented vectors are generated on-the-fly

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.3. Single/Multi-category Weighting

- **Single category** augmented weights:

- Formally: $w_E(t_i) = TF(t_i, E) \times IEF_{Cat}(t_i, D)$

- where $IEF_{Cat}(t_i, D) = \log \frac{N_{Cat}}{EF_{Cat}(t_i, C)}$

- **Multi-category** augmented weights:

- Formally: $w_E(t_i) = TF(t_i, E) \times IEF_{MCat}(t_i, D)$

- where $IEF_{MCat}(t_i, D) = \log \frac{\sum_{Cat \in M} N_{Cat}}{\sum_{Cat \in M} EF_{Cat}(t_i, C)}$

⇒ Considering all categories comes down to computing typical TF-IEF weights

- Considering all indexing nodes in the document as a whole

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

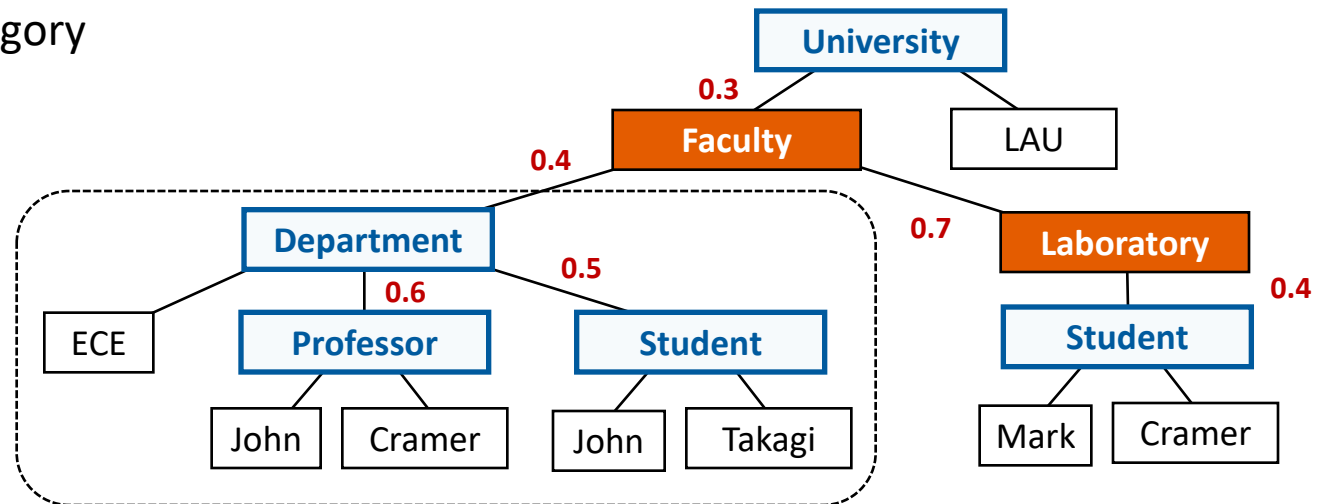
Document tree D1

Example: Single category

User is interested
in *professors* and *students*
in academic *departments*:



Category: sub-tree
rooted at *Department*



Base vectors:

	John	Cramer	Takagi
$\vec{V}_{Prof}$	$1 \times \log(3/2)$	$1 \times \log(3/1)$	0
$\vec{V}_{Stud1}$	$1 \times \log(3/2)$	0	$1 \times \log(3/1)$

NOTE: If only leaf nodes were considered as indexing nodes

⇒ Term “John” would have been useless in indexing nodes of category *Department*

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

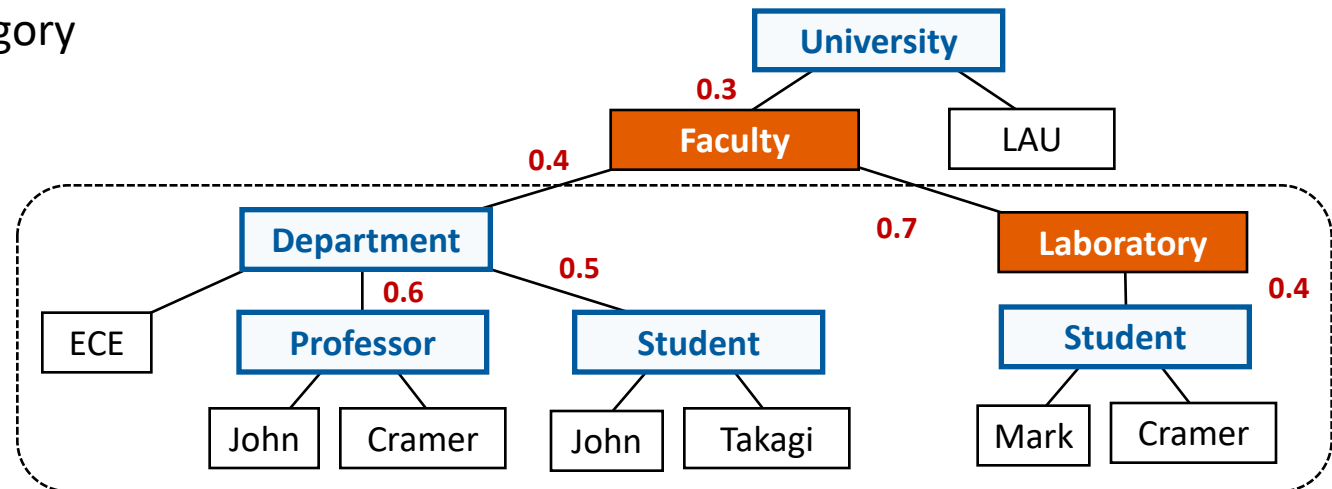
Document tree D1

Example: Multi-category

User is interested
In *professors* and *students*
in *departments*
and *laboratories*



Categories: sub-trees
rooted at *Department*
and *Laboratory*



Base vectors:

	John	Cramer	Takagi	Mark
$\vec{V}_{Prof}$	$1 \times \log(4/2)$	$1 \times \log(4/2)$	0	0
$\vec{V}_{Stud1}$	$1 \times \log(4/2)$	0	$1 \times \log(4/1)$	0
$\vec{V}_{Stud2}$	0	$1 \times \log(4/2)$	0	$1 \times \log(4/1)$

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.4. Term Context Model

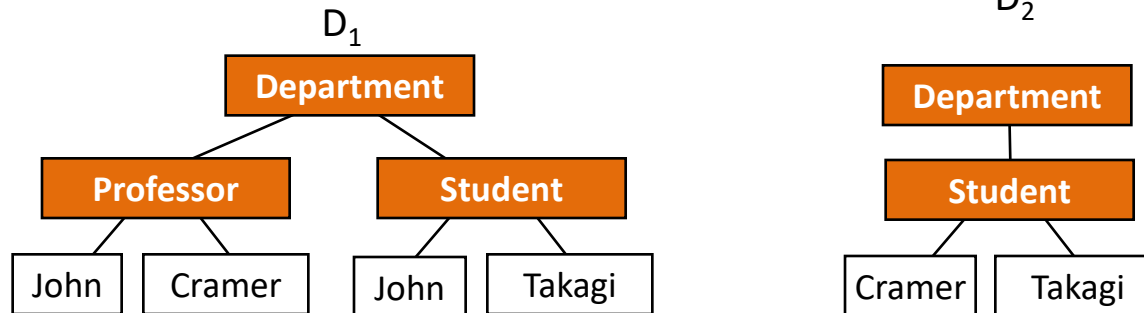
- Extending the vector space model by:
 - Replacing the basic indexing units, i.e. terms $\mathbf{t}_i$, by pairs of the form $(\mathbf{t}_i, \mathbf{c}_i)$
 - Where each term $\mathbf{t}_i$ is qualified by the context $\mathbf{c}_i$ in which it appears [11]
- The context $\mathbf{c}_i$ of appearance of a term $\mathbf{t}_i$ is the root path of $\mathbf{t}_i$
 - Path for navigating the hierarchical structure of the document from the reference (root) node to the node in which term $\mathbf{t}_i$ occurs
- Weights are computed for each $(\mathbf{t}_i, \mathbf{c}_i)$ pair, of the form $\mathbf{w}_D(\mathbf{t}_i, \mathbf{c}_i)$
 - Performing **TF-IDF** computations accordingly
 - ⇒ Processing is done at the document level
 - Not at the element level, in contrast with previous semi-structured approaches

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example: Term context model

Consider two document trees:



	(John, Dept/Prof)	(Cramer, Dept/Prof)	(John, Dept/Stud)	(Takagi, Dept/Stud)	(Cramer, Dept/Stud)
$\vec{V}_{D1}$	$1 \times \log(2/1)$	$1 \times \log(2/1)$	$1 \times \log(2/1)$	$1 \times \log(2/2)$	0
$\vec{V}_{D2}$	0	0	0	$1 \times \log(2/2)$	$1 \times \log(2/1)$

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.4. Term Context Model

- In addition to the term/context representation
 - The authors **extend the** term/context **weighting scheme**
 - By accounting not only for exact term in context matching
 - But also for context resemblance

$$\forall \mathbf{w}(\mathbf{t}_i, \mathbf{c}_i) = 0, \text{ extended } \mathbf{w}(\mathbf{t}_i, \mathbf{c}_i) = \underset{(\mathbf{t}_i, \mathbf{c}_j) \in D}{\text{Max}} \left(\mathbf{w}_D(\mathbf{t}_i, \mathbf{c}_j) \times \text{Sim}_{\text{ED}}(\mathbf{c}_i, \mathbf{c}_j) \right)$$

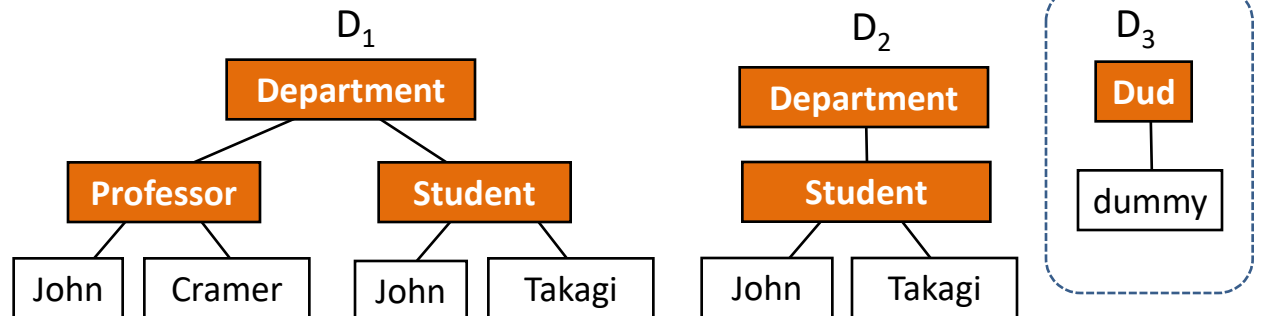
- Where $\text{Sim}_{\text{ED}}(\mathbf{c}_i, \mathbf{c}_j)$ measures the similarity between context paths $\mathbf{c}_i$ and $\mathbf{c}_j$
 - Computed based on string **edit distance** (ED, cf. Ch. 3)
 - Where paths $\mathbf{c}_i$ and $\mathbf{c}_j$ are represented as strings of node labels

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example: Term context model

Consider three document trees:



$\vec{V}_{D1}$	(John, Dept/Prof)	(Cramer, Dept/Prof)	(John, Dept/Stud)	(Takagi, Dept/Stud)
	$1 \times \log(3/1)$	$1 \times \log(3/1)$	$1 \times \log(3/2)$	$1 \times \log(3/2)$

$\vec{V}_{D2}$	(John, Dept/Stud)	(Takagi, Dept/Stud)
	$1 \times \log(3/2)$	$1 \times \log(3/2)$

$\vec{V}_{D3}$	(dummy, Dud)
	$1 \times \log(3/1)$

3. Extending IR Techniques to Semi-structured Data

When comparing $\vec{V}_{D1}$ and $\vec{V}_{D2}$:

	(John, Dept/Prof)	(Cramer, Dept/Prof)	(John, Dept/Stud)	(Takagi, Dept/Stud)
$\vec{V}_{D1}$	$1 \times \log(3/1)$	$1 \times \log(3/1)$	$1 \times \log(3/2)$	$1 \times \log(3/2)$
$\vec{V}_{D2}$	0	0	$1 \times \log(3/2)$	$1 \times \log(3/2)$

Given $\text{Sim}_{ED}(c_i, c_k) = \frac{1}{1 + ED(c_i, c_k)}$ using (Wagner-Fisher, 1974)'s ED algorithm to compare contexts represented as strings

For instance: $ED(\text{Dept/Prof}, \text{Dept/Stud}) = 1$, the cost of updating *Prof* into *Stud*

$\Rightarrow \text{Sim}_{ED}(\text{Dept/Prof}, \text{Dept/}) = 0.5$

No need to update weights of $\vec{V}_{D1}$ since all weights are $\neq 0$

Updating weights of $\vec{V}_{D2}$:

Considering term: $t_i = \text{John}$ in context $c_i = \text{Dept/Prof}$:

$$\begin{aligned}
 \hookrightarrow w(t_i, c_i) &= \text{Max}_{(t_i, c_j) \in D} (w_D(t_i, c_j) \times \text{Sim}_{ED}(c_i, c_j)) \\
 &= \text{Max}_{(t_i, c_j) \in D} \left(\begin{array}{l} (0 \times \text{Sim}_{ED}(\text{Dept/Prof}, \text{Dept/Prof})), \\ (\log(3/2) \times \text{Sim}_{ED}(\text{Dept/Prof}, \text{Dept/Stud})) \end{array} \right) = \log(3/2) \times 0.5
 \end{aligned}$$

3. Extending IR Techniques to Semi-structured Data

When comparing $\vec{V}_{D1}$ and $\vec{V}_{D2}$:

	(John, Dept/Prof)	(Cramer, Dept/Prof)	(John, Dept/Stud)	(Takagi, Dept/Stud)
$\vec{V}_{D1}$	$1 \times \log(3/1)$	$1 \times \log(3/1)$	$1 \times \log(3/2)$	$1 \times \log(3/2)$
$\vec{V}_{D2}$	0	0	$1 \times \log(3/2)$	$1 \times \log(3/2)$

Updating weights of $\vec{V}_{D2}$:

Considering term: $t_i = \text{Cramer}$ in context Dept/Prof:

$$\hookrightarrow \mathbf{w}(t_i, c_i) = \mathbf{Max}_{(t_i, c_j) \in D} ((0 \times \mathbf{Sim}_{ED}(\text{Dept/Prof}, \text{Dept/Prof}))) = 0$$

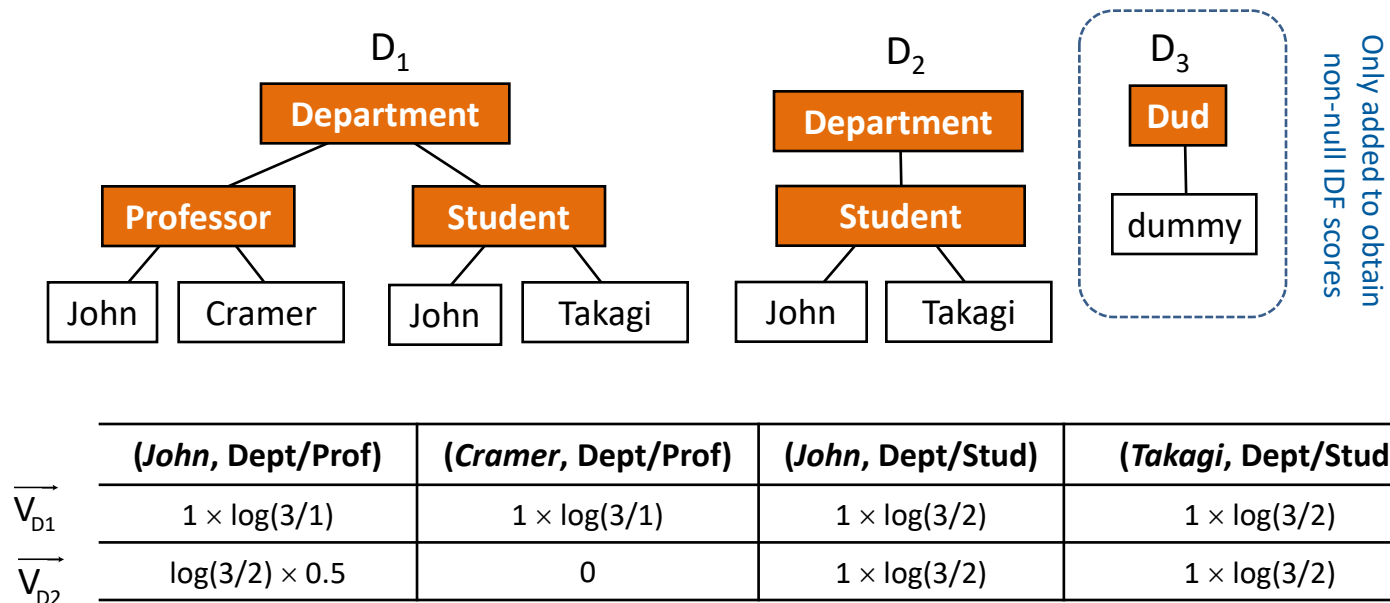
- No need to update remaining weights since they have non-null values

Updated vectors:

	(John, Dept/Prof)	(Cramer, Dept/Prof)	(John, Dept/Stud)	(Takagi, Dept/Stud)
$\vec{V}_{D1}$	$1 \times \log(3/1)$	$1 \times \log(3/1)$	$1 \times \log(3/2)$	$1 \times \log(3/2)$
$\vec{V}_{D2}$	$\log(3/2) \times 0.5$	0	$1 \times \log(3/2)$	$1 \times \log(3/2)$

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data



As a result: $\text{Sim}_{\text{ECosine}}(\vec{V}_{D1}, \vec{V}_{D2}) = \frac{\log(3) \times \log(3/2) \times 0.5 + 2 \times \log(3/2)^2}{|\vec{V}_{D1}| \times |\vec{V}_{D2}|}$

Yet, traditional cosine would have produced: $\text{Sim}_{\text{ECosine}}(\vec{V}_{D1}, \vec{V}_{D2}) = \frac{2 \times \log(3/2)^2}{|\vec{V}_{D1}| \times |\vec{V}_{D2}|}$

⇒ Missing inter-context similarities with term *John*

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

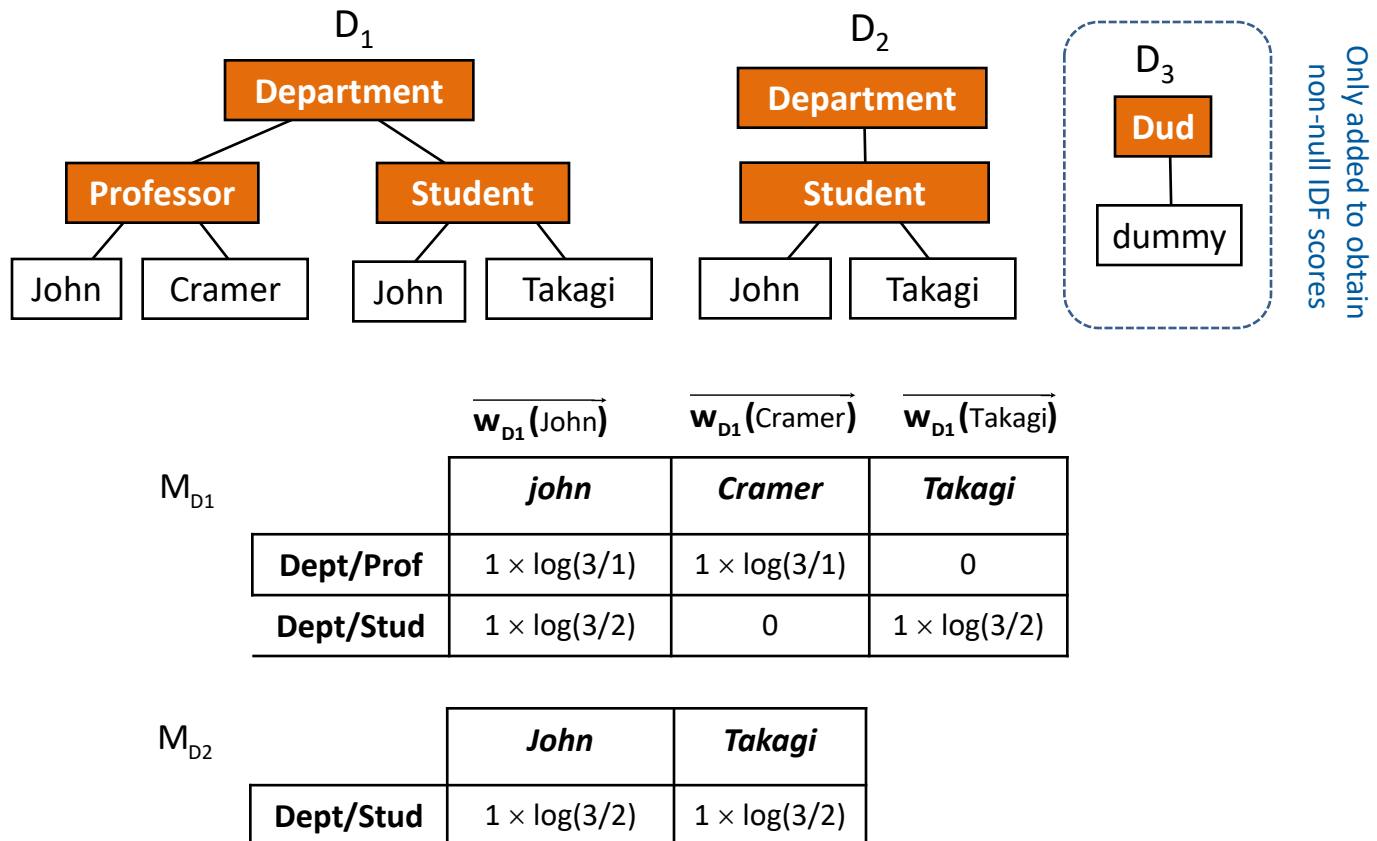
3.5. Matrix Model

- Extending/transforming term/context vector representation model
 - To [matrix representation model](#) [12]
- In the vector space model, a weight $w_d(t_i)$ is expressed by a real number (e.g. *TF-IDF* score) specifying the distribution of a term t_i for the entire document
 - In the matrix model, the weight of a term is expressed by a vector $\overrightarrow{w_d(t_i)}$
 - Reflecting the distribution of the term t_i in the document paths
 - W.r.t. to each path occurring in the document collection

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example: Matrix representation model



Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

3.5. Matrix Model

- Comparing two documents comes down to
 - Comparing corresponding matrices
 - Using matrix similarity measures, e.g., extended **cosine matrix similarity**

$$\begin{aligned}
 \text{Sim}_{\text{Cosine}}(\mathbf{M}_Q, \mathbf{M}_D) &= \frac{\sum_{i=1 \dots U} \sum_{j=1 \dots V} \mathbf{w}_Q(t_i, p_j) \times \mathbf{w}_D(t_i, p_j)}{|\mathbf{M}_Q| \times |\mathbf{M}_D|} \\
 &= \frac{\sum_{i=1 \dots U} \sum_{j=1 \dots V} \mathbf{w}_Q(t_i, p_j) \times \mathbf{w}_D(t_i, p_j)}{\sqrt{\sum_{i=1 \dots N} \sum_{j=1 \dots V} \mathbf{w}_Q(t_i, p_j)^2} \times \sqrt{\sum_{i=1 \dots N} \sum_{j=1 \dots V} \mathbf{w}_D(t_i, p_j)^2}} \in [-1, 1]
 \end{aligned}$$

Ch. 7. Structure and Content Similarity

3. Extending IR Techniques to Semi-structured Data

Example:

M_{D1}	<i>john</i>	<i>Cramer</i>	<i>Takagi</i>	
	$1 \times \log(3/1)$	$1 \times \log(3/1)$	0	Dept/Prof
	$1 \times \log(3/2)$	0	$1 \times \log(3/2)$	Dept/Stud

M_{D2}	<i>john</i>	<i>Cramer</i>	<i>Takagi</i>	
	0	0	0	Dept/Prof
	$1 \times \log(3/2)$	0	$1 \times \log(3/2)$	Dept/Stud

$$\text{Sim}_{\text{Cosine}}(M_{D1}, M_{D2}) = \frac{\log(3/2) \times \log(3/2) + \log(3/2) \times \log(3/2)}{|M_Q| \times |M_D|} = \frac{2 \times \log(3/2)^2}{|M_Q| \times |M_D|} \in [-1, 1]$$

⇒ Comes down to the same result obtained with term/content model
 - When using **traditional cosine** vector similarity measure

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- Processing keyword-based queries is a fundamental problem in IR
 - Main premise of keyword-based querying:
 1. The user issues a keyword query containing a bunch of terms
 2. Then, documents containing the query terms are retrieved
 - Ranked following their relevance (similarity) w.r.t. the query
- The brute force approach is to sequentially scan documents
 - Searching for terms that map to the query keywords
 - **Problem: Very inefficient** for big document collections
- **Solution:** Use some sort of **indexing** to speed up the search process

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- The foremost index structure used with text search is: **inverted index**
 - Associating terms in text with a list of pointers to the documents containing the terms: a list of **(term, docIDs[])**

ID	Textual description (in IR/vector representation)
D1	<u>When a Stranger Calls</u> (2006): A young high school student babysits for a very rich family with a car ...
D2	<u>Days of Thunder</u> (1990): Cole Trickle is a young racer from California with years of experience in open-wheel racing winning championships in Sprint car racing...
D3	<u>Sound of Music, The</u> (1965): Maria had longed to be a nun since she was a young girl...
...	...



Inverted index

Term	Doc IDs
Car	D1, D2
Call	D1
Day	D2
Sound	D3
Music	D3
...	...

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- The inverted index is queried with every term within the user query
 - Identifying documents that contain the query terms in **one search**
 - Using either **intersection** (contains all terms) or **union** (contains any term) operators
- For instance, given keyword query **q = "sound of music"**
 - The index would return document **D3** as a an answer

ID	Textual description (in IR/vector representation)
D1	<u>When a Stranger Calls</u> (2006): A young high school student babysits for a very rich family with a car...
D2	<u>Days of Thunder</u> (1990): Cole Trickle is a young racer from California with years of experience in open-wheel racing winning championships in Sprint car racing...
D3	<u>Sound of Music, The</u> (1965): Maria had longed to be a nun since she was a young girl...
...	...



Term	Doc IDs
Car	D1, D2
Call	D1
Day	D2
Sound	D3
Music	D3
...	...

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- The inverted index can be augmented to include term weights
 - The index structure is straightforwardly extended to include, with each docID, the corresponding term weight: ***(term, docIDs_termWeights[])***
 - For instance, $w_{D1}(\text{"car"}) = \text{TF}(t_i, D_1) \times \text{IDF}(t_i, C) = 1 \times \log(3/2) = \log(3/2)$

ID	Textual description (in IR/vector representation)
D1	<u>When a Stranger Calls</u> (2006): A young high school student babysits for a very rich family with a car...
D2	<u>Days of Thunder</u> (1990): Cole Trickle is a young racer from California with years of experience in open-wheel racing winning championships in Sprint car racing...
D3	<u>Sound of Music, The</u> (1965): Maria had longed to be a nun since she was a young girl...
...	...

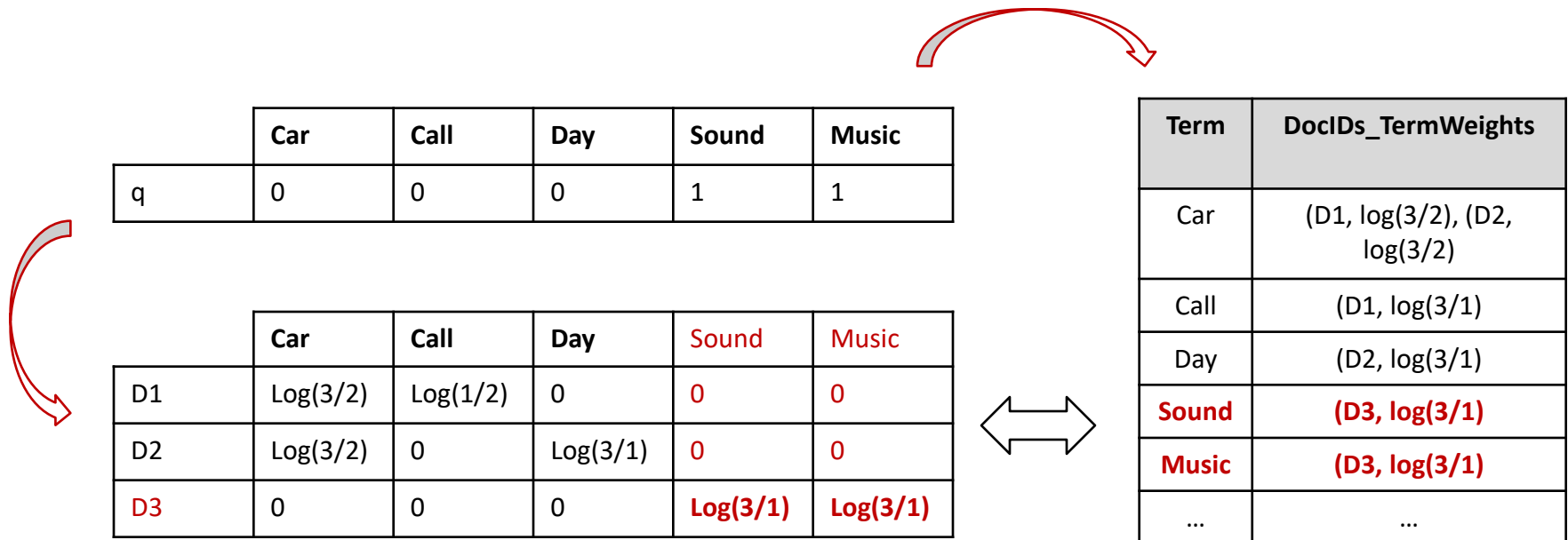


Term	DocIDs_Term Weights
Car	(D1, log(3/2), (D2, log(3/2))
Call	(D1, log(3/1))
Day	(D2, log(3/1))
Sound	(D3, log(3/1))
Music	(D3, log(3/1))
...	...

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

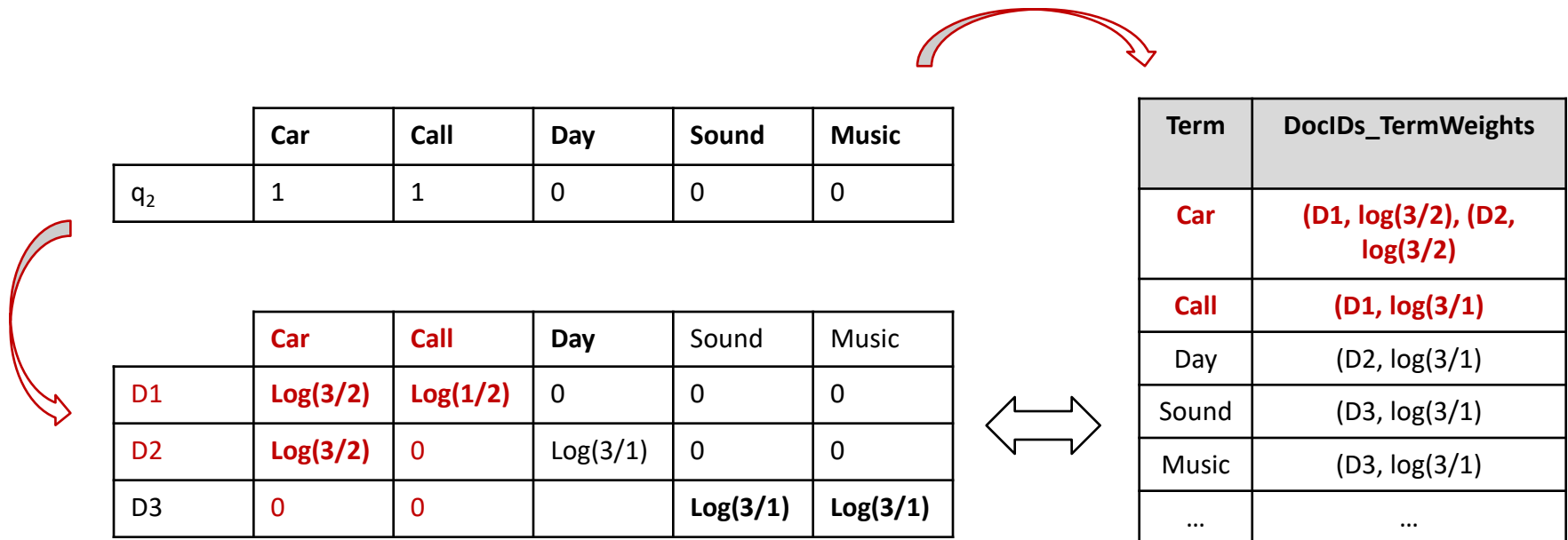
- The inverted index can be augmented to include term weights
 - As a result, queries can be executed using similarity-based operators
- For instance, given keyword query **q = “Sound of music”**
 - $\text{Sim}_{\text{Cos}}(\mathbf{q}, \mathbf{D3}) > \text{both } \text{Sim}_{\text{Cos}}(\mathbf{q}, \mathbf{D1}) = \text{Sim}_{\text{Cos}}(\mathbf{q}, \mathbf{D2}) = 0$



Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- The inverted index can be augmented to include term weights
 - As a result, queries can be executed using similarity-based operators
- For instance, given keyword query $q_2 = \text{"Call for car"}$
 - $\text{Sim}_{\text{Cos}}(q_2, D1) > \text{Sim}_{\text{Cos}}(q_2, D2) > \text{Sim}_{\text{Cos}}(q_2, D3)=0$



Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- ◆ Extending inverted index model to consider **multi-attribute data records**
 - Collection of data organized as a set of data records (e.g., a table in RDBs)
 - Each data record consists of a set of values for a **set of attributes**

ID	Title	Date	Plot
O1	When a Stranger Calls	2006	A young high school student babysits for a very rich family...
O2	Days of Thunder	1990	Cole Trickle is a young racer from California with years of experience...
O3	Sound of Music, The	1965	Maria had longed to be a nun since she was a young girl, yet when...
...	...		

- Extended inverted indexes to process multi-attribute records:
 1. List of **(term, (recordID, attribute)[])**
 2. List of **((term, attribute), recordID[])**

4. Underlying Indexing Techniques

Multi-attribute text data collection

ID	Title	Date	Plot
D1	When a Stranger Calls	2006	A young high school student babysits for a very rich family...
D2	Days of Thunder	1990	Cole Trickle is a young racer from California with years of experience...
D3	Sound of Music, The	1965	Maria had longed to be a nun since she was a young girl, yet when...
...	...		

Index 1: $(term, (recordID, att)[])$

Term	(RecordID, Attribute) []
"Car"	(D1, Plot), (D2, Plot)
"Zen"	(D1, Plot)
"Light"	(D1, Plot)
"Sound"	(D1, Plot), (D3, Title), (D3, Plot),
...	...

Index 2: $((term, attribute), recordID[])$

(term, attribute)	recordID[]
("Car", Title)	-
("Car", Plot)	D1, D2
("Sound", Title)	D3
("Sound", Plot)	D1, D3
...	...

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

◆ Extending inverted index model to consider **semi-structured data**

– Same models adopted with multi-attribute records

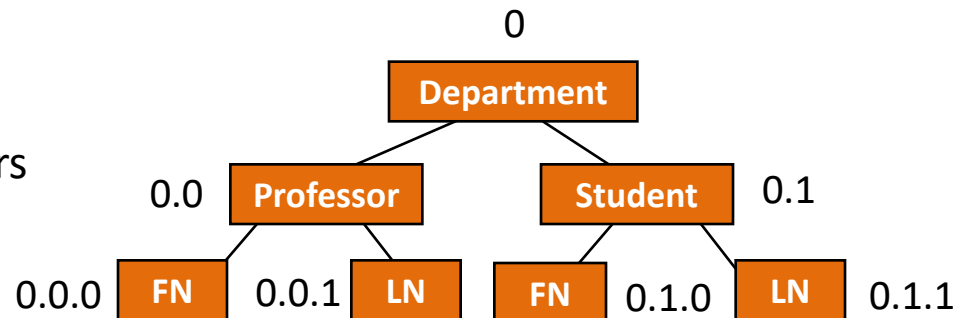
- Replacing attributes with element identifiers:

1. List of **(term, (docID, elementID)[])**

2. List of **((term, elementID), docID[])**

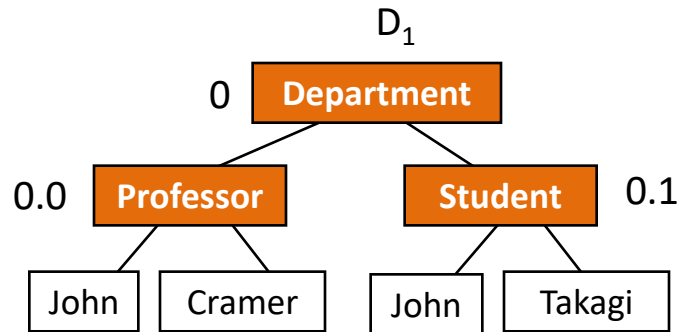
- Element can be identified in different ways, e.g.:

- Root paths contexts
- **Dewey numbers**
- Pre/post-order numbers



Ch. 7. Structure and Content Similarity

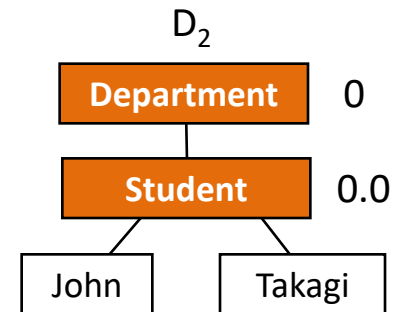
4. Underlying Indexing Techniques



Index 1: $(term, (docID, eleID)[])$

Term	(docID, EleID) []
"John"	(D1, 0.0), (D1, 0.1), (D2, 0.0)
"Cramer"	(D1, 0.0)
"Takagi"	(D1, 0.1), (D2, 0.0)

↳ Using Dewey numbers for eleIDs



Index 2: $((term, eleID), docID[])$

(term, eleID)	docID[]
("John", Dept/Prof)	D1
("John", Dept/Stud)	D1, D2
("Cramer", Dept/Prof)	D1
("Takagi", Dept/Stud)	D1, D2

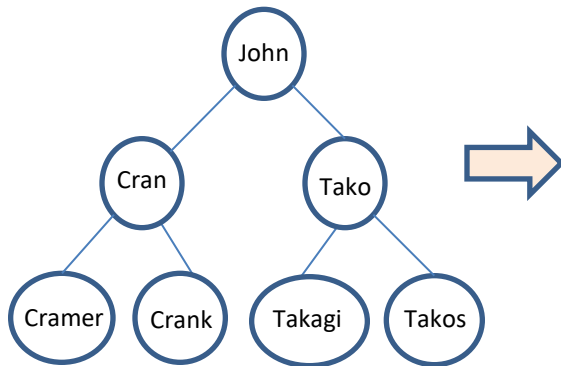
↳ Using root paths for eleIDs

Ch. 7. Structure and Content Similarity

4. Underlying Indexing Techniques

- ◆ In addition, the inverted index itself is indexed using typical techniques
 - ◆ E.g., **Binary tree**, **B-Tree**, **B+ tree**, **2-3 tree**, etc.

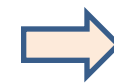
Binary tree index



Inverted Index: $(term, (docID, eleID)[\])$

Term	(docID, EleID) []
"Cramer"	(D1, 0.0)
"John"	(D1, 0.0), (D1, 0.1), (D2, 0.0)
"Takagi"	(D1, 0.1), (D2, 0.0)

↳ Using Dewey numbers for eleIDs



**IR (VSM)
Model**

Ch. 7. Structure and Content Similarity

5. From IR to AI-enabled RAG

- **Traditional Information Retrieval (IR)**
 - Indexing, ranking, querying using **VSM**, **TF-IDF**, **context paths**, etc.
 - Returns **documents** that are relevant w.r.t. a user query
- **Large Language Models (LLMs)**
 - Provide fluent natural language answers
 - But rely on learned, static internal parameters
 - Prone to **hallucinations**, outdated knowledge
- **Motivation for Retrieval Augmented Generation (RAG)**
 - Combine **IR precision** + **LLM generation**
 - Ensure answers are grounded in **retrieved and relevant** content
 - Leverage semi-structured data, structured metadata, or raw text

Retrieval provides *content*, generation provides *language*

Ch. 7. Structure and Content Similarity

5. From IR to AI-enabled RAG

RAG Pipeline Overview

1. Query Processing

- User issues a natural-language question
- The system encodes it into a **vector representation** (similar to VSM)

2. Retrieval Component (IR)

- Searches in an **inverted index** or **semi-structured index**
- Uses **similarity search** (cosine / ANN / semantic similarity)
- Retrieves top-k relevant **chunks / nodes / passages**

3. Augmentation Step

- Retrieved content is injected into the LLM prompt
 - Functions similarly to **context weighting**, **category restriction**, or **term-in-context indexing**

4. Generation

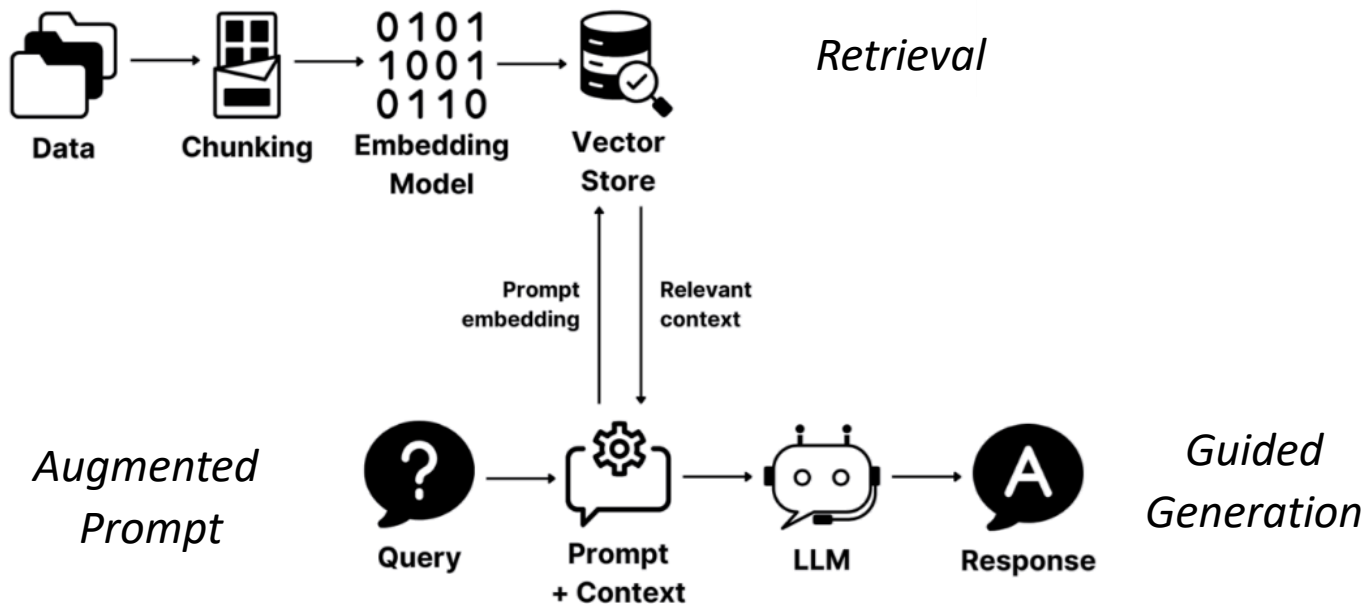
- LLM generates an answer **grounded** in the retrieved evidence
 - Reduces hallucination; increases domain-specific accuracy

Ch. 7. Structure and Content Similarity

5. From IR to AI-enabled RAG

RAG Pipeline Overview

RAG = Retrieval + Augmented Prompt + Guided Generation



Ch. 7. Structure and Content Similarity

5. From IR to AI-enabled RAG

RAG on Semi-Structured Data

- **Semi-structured data in RAG**
 - XML/JSON/RDF trees can be indexed as granular “knowledge chunks”
 - **Category-based retrieval** aligns with retrieval filters (facets, metadata)
 - **Context-aware similarity** improves retrieval quality for LLMs
 - Matrix/path models parallel modern **graph-based RAG** extensions
- **RAG is a natural evolution of IR applied to generative models**
 - Building on structural & content similarity...
- **Benefits of integrating Semi-structured IR with RAG**
 - Precise, structure-aware selection of relevant content
 - Higher factual accuracy in generated outputs
 - Support for domain-specific ontologies and controlled vocabularies

Ch. 7. Structure and Content Similarity

Wrapping up

- **Information retrieval** (*IR*) is a branch of informatics concerned with the acquisition, organization, storage, search and selection of **unstructured information**
 - In comparison with database systems which handle structured information
- IR techniques have been **extended** to handle **semi-structured data**
 - Introducing dedicated representation models
 - **Indexing nodes**
 - **Structural weights**
 - **Augmented vectors**
 - **Single/multi-category retrieval**
 - **Context paths**
 - **Matrix model**
- Underlying indexing techniques

Ch. 7. Structure and Content Similarity

Questions?



Ch. 7. Structure and Content Similarity

Existing IR Open Source Platforms

- ElasticSearch, <https://www.elastic.co/elasticsearch>
- Apache Solr, <https://lucene.apache.org/solr/>
- Apache Leucene Library, <https://lucene.apache.org/>

Exercises and Tutorials

- Provided by the instructor during lecture sessions



Ch. 7. Structure and Content Similarity

Main References

1. Stefanie Nadig, Martin Braschler, Kurt Stockinger: Database Search vs. Information Retrieval: A Novel Method for Studying Natural Language Querying of Semi-Structured Data. LREC 2020: 1772-1779
2. Liang Zhang, Bingpeng Ma, Guorong Li, Qingming Huang, Qi Tian: Generalized Semi-supervised and Structured Subspace Learning for Cross-Modal Retrieval. IEEE Trans. Multim. 20(1): 128-141 (2018)
3. Adilbek O. Erkimbaev, Vladimir Y. Zitserman, Georgii A. Kobzev, Andrey V. Kosinov: Standardization of Storage and Retrieval of Semi-structured Thermophysical Data in JSON-documents Associated with the Ontology. DAMDID/RCDL 2017: 219-224
4. Sana Fakhfakh, Mohamed Tmar, Walid Mahdi: Multimedia Retrieval based on Geometric Distance in Semi-structured Document. WEBIST (1) 2014: 220-225
5. G. Salton and M.J. McGill. Introduction to Modern Information Retrieval, McGraw-Hill, Tokyo, 1983.
6. Tsatsaronis, George; Vicky Panagiotopoulou (2009-04-02), A Generalized Vector Space Model for Text Retrieval Based on Semantic Relatedness , ACM EACL.
7. S. Deerwester, S. Dumais, G. Furnas, T. Landauer and R. Harshman. Indexing by Latent Semantic Analysis. Journal of the American Society for Information Science, 41(6), (1990) pp. 391-407.
8. N. Fuhr. Probabilistic models in information retrieval. The Computer Journal 35, 3, (1992) pp. 243-255.
9. Gianni Amati, Cornelis Joost Van Rijsbergen, Probabilistic models of information retrieval based on measuring the divergence from randomness, ACM Transactions on Information Systems (TOIS) Volume 20 Issue 4, October 2002 Pages 357-389
10. N. Fuhr and K. Großjohann. XIRQL: A Query Language for Information Retrieval. In: Proceedings of ACM-SIGIR, New Orleans, (2001) pp. 172-180.